

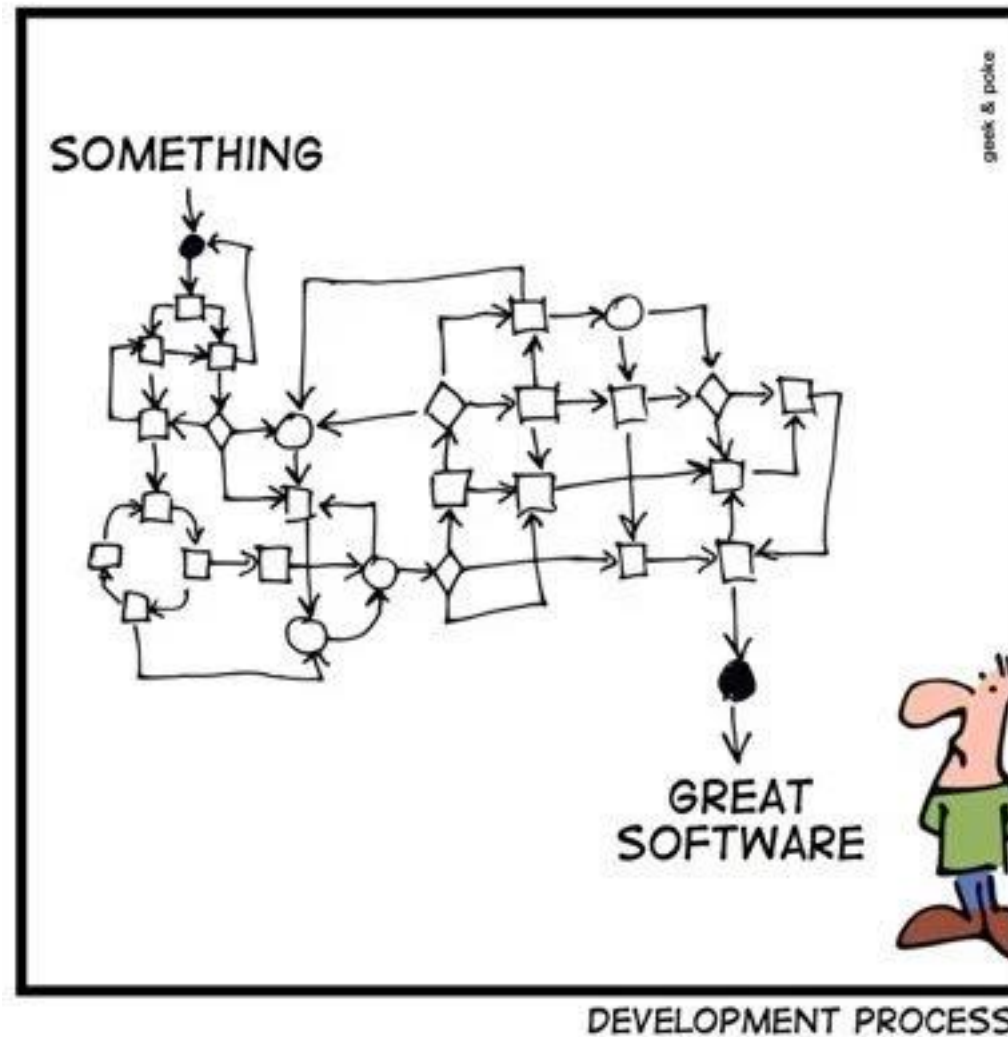
Developer Operations (devops)

Building Software

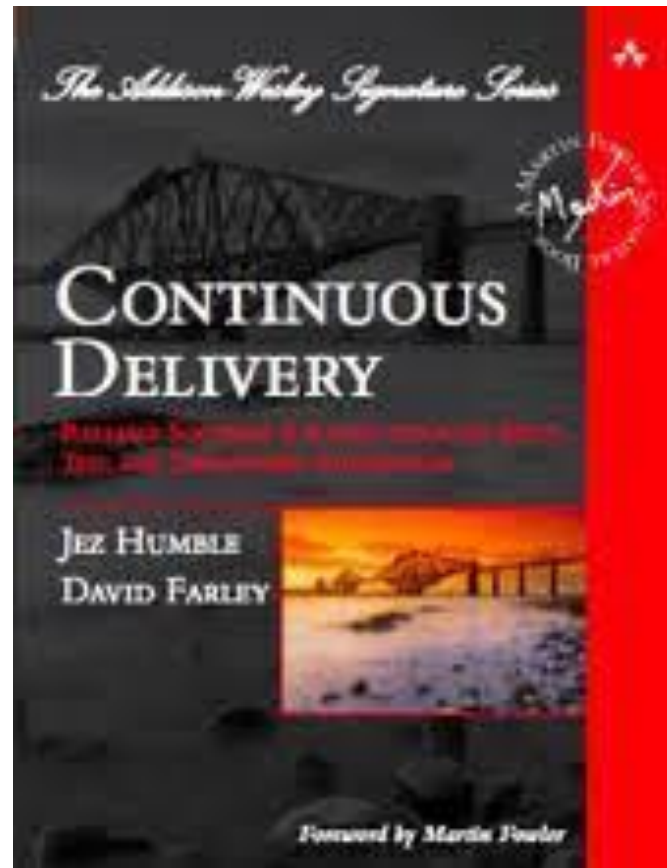


Building Software

SIMPLY EXPLAINED



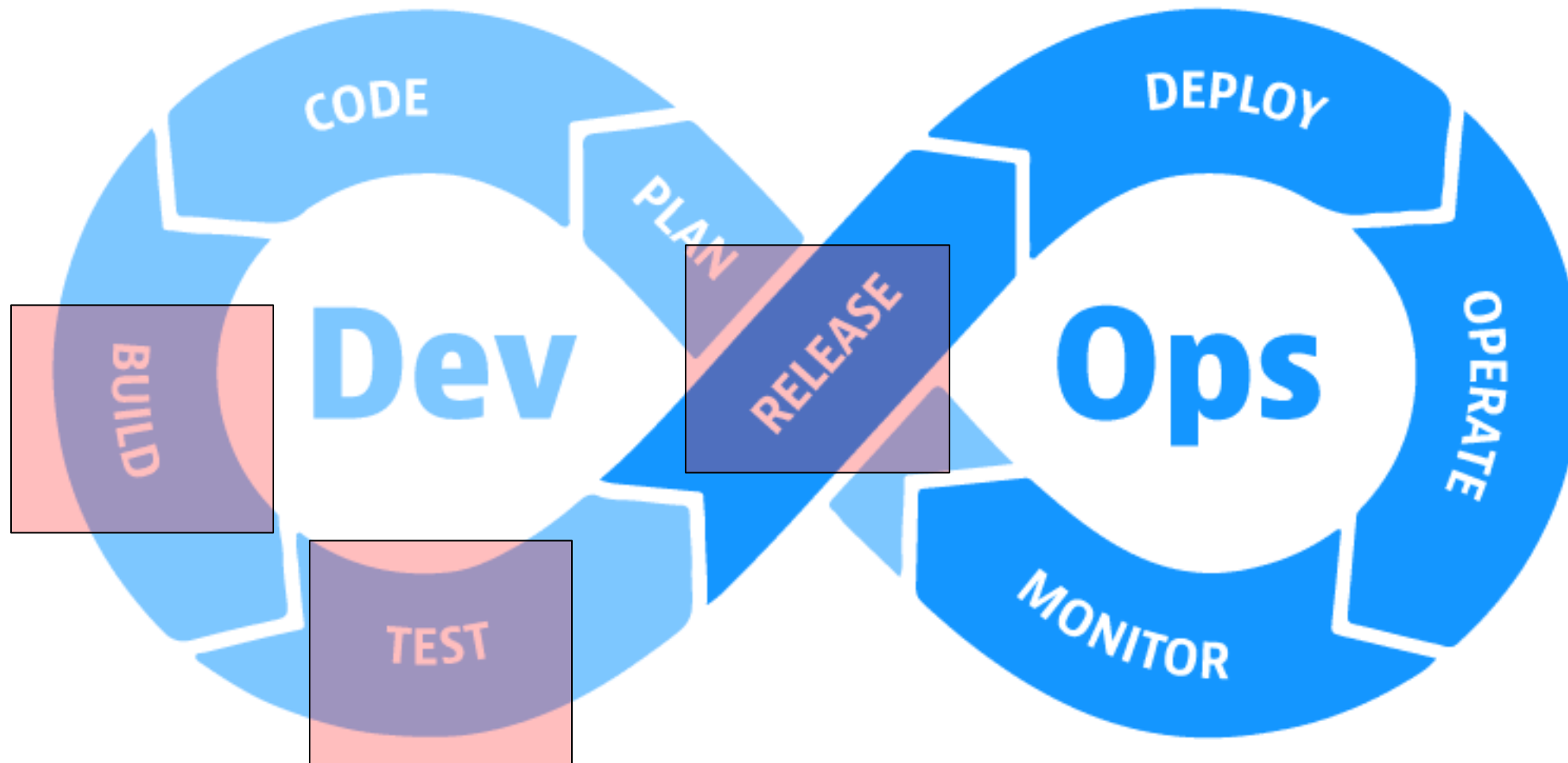
Building Software, Principles of Software Delivery



1. *Create a Repeatable, Reliable Process for Releasing Software*
2. *Automate Almost Everything*
3. *Keep Everything in Version Control*
4. *If It Hurts, Do It More Frequently, and Bring the Pain Forward*
5. *Build Quality In*
6. *Done Means Released*
7. *Everybody Is Responsible for the Delivery Process*
8. *Continuous Improvement*

We will use Gitlab CI for creating a suitable release process satisfying these points.

Build and Release



What tests do you want to continuously run?

What analysis do you want to continuously perform?

What is a release?

Release != Deployments

Separation of Concerns:

- Different Domains / Tools for generating artifact and deploying them

Quick, painless builds:

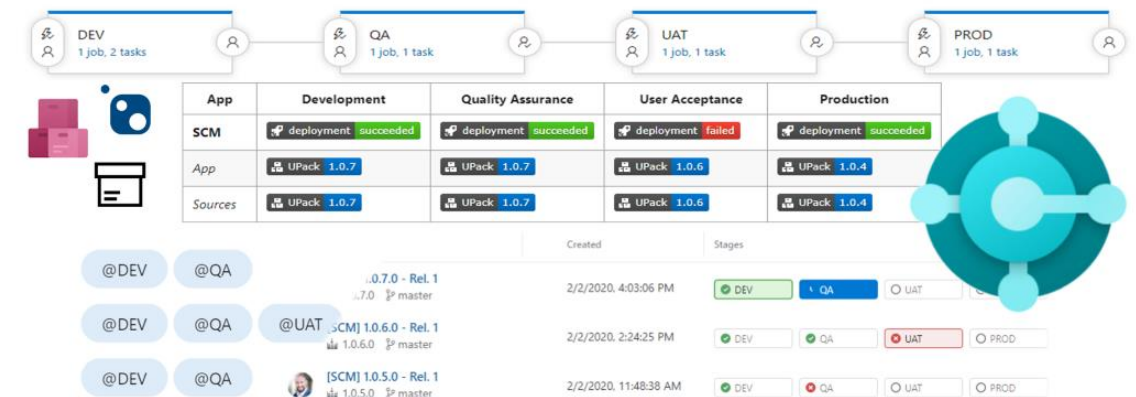
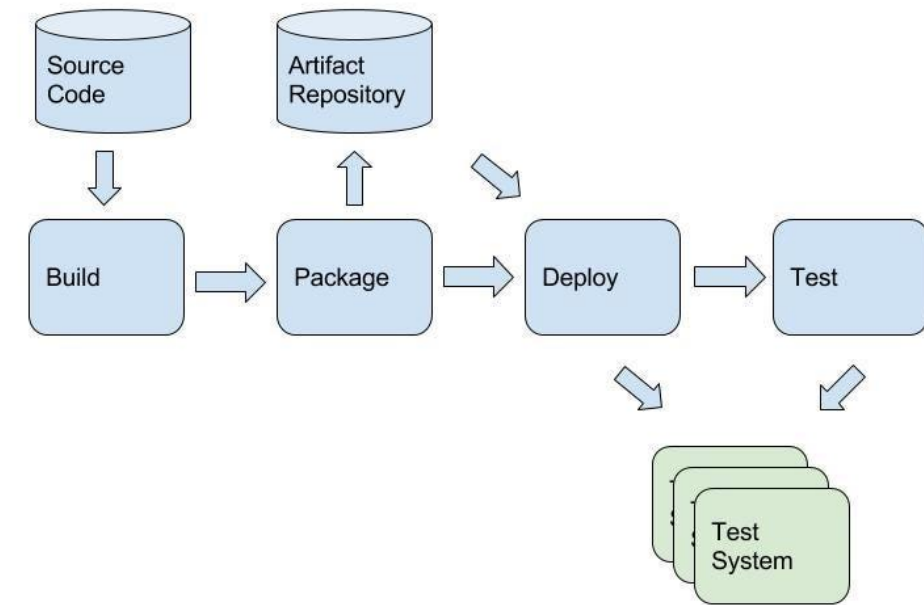
- Builds (and thereby releases) should be automated and painless just generating artifacts

Independent deployable and immutable artifacts:

- Deployment Units should be independent from environment (stage)

Idempotent and scalable tests:

- By separating deployment from releasing, scalable tests on the same artifact become possible



What is a release in frameworks?

Maven

- Identified by fixed version in GAV(C)
- Unique with respect to GAV(C)
- Not to be modified again
 - Seen as “stable”
 - Protected for overriding artifact

Otherwise, transitive dependencies will break

For continuous development,
please use “-SNAPSHOT”-suffix

Docker / OCI

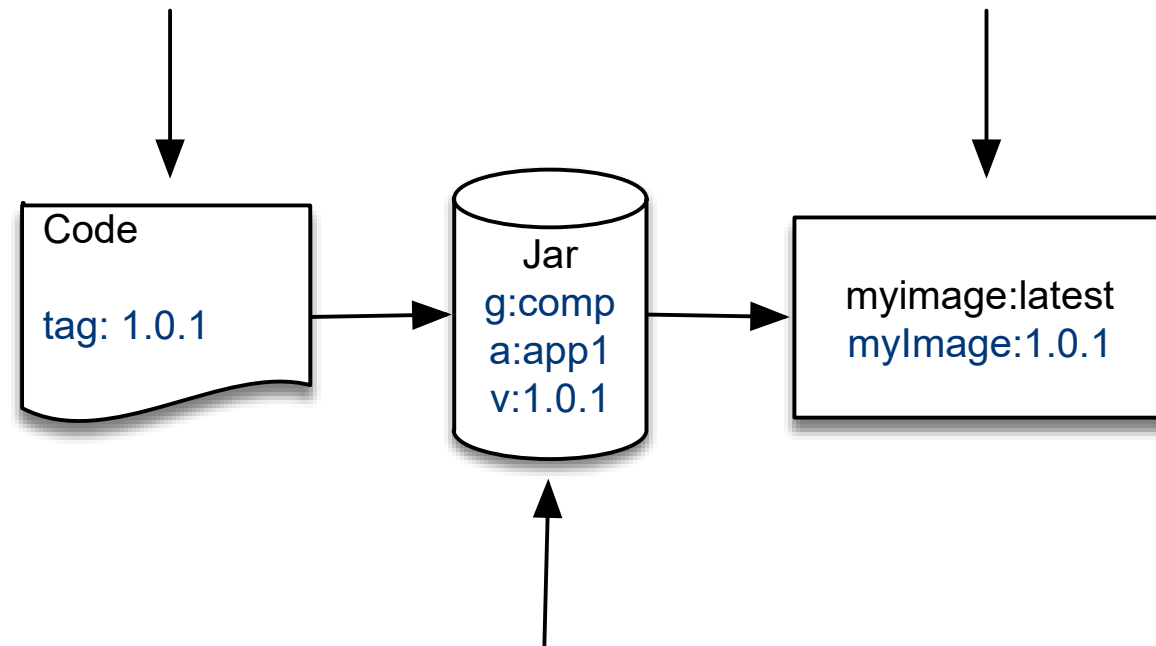
- Identified by Tags
- Unique with respect of NAME:TAG
- Can be overridden, seldomly overridden
- No danger of breaking transitive dependencies -> Images are self-contained

For continuous development,
please use the tag “latest”

Releases can be complex...

```
>git tag 1.0.1
>git push -tags
```

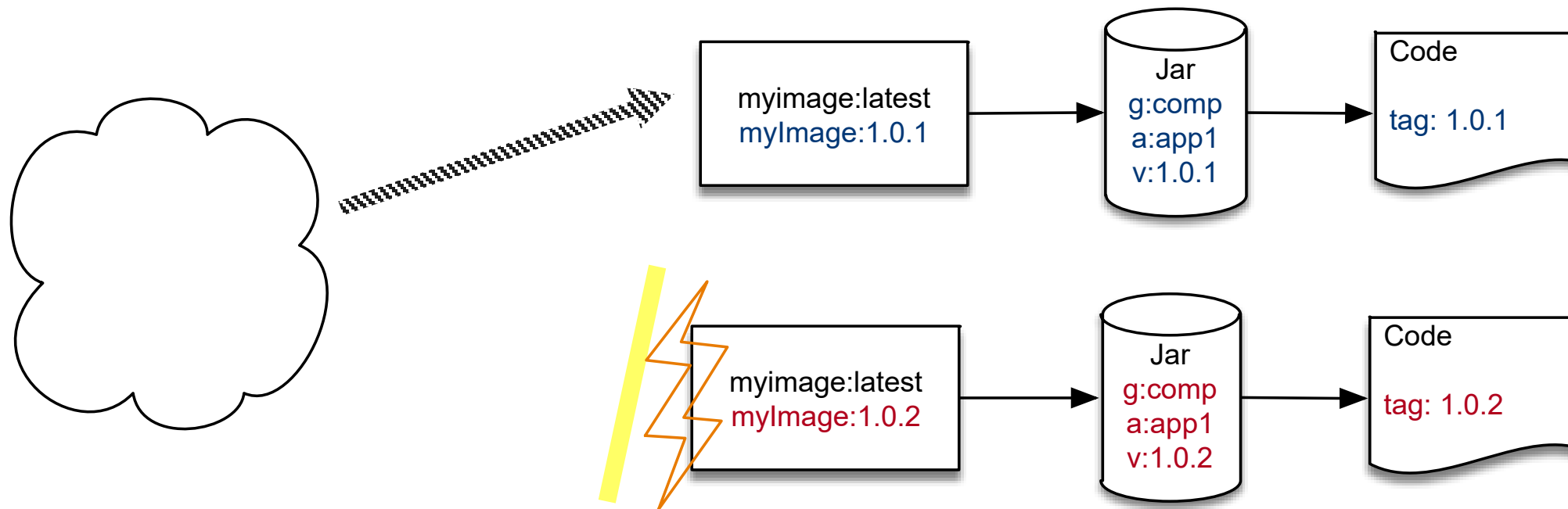
```
>docker tag myImage:1.0.1
>docker push myImage:1.0.1
```



```
>mvn release:perform -DreleaseVersion=1.0.1
```

What is a problem when releasing on a CI?

Why is regular releasing important?

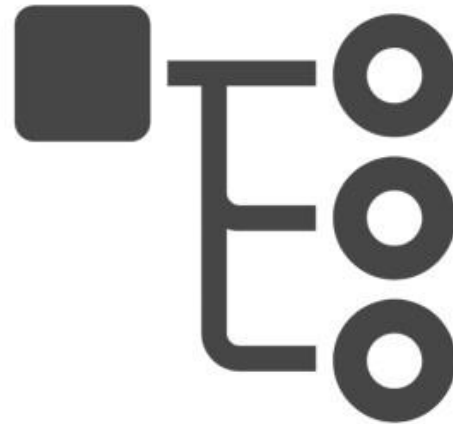


Why?

Three pillars of observability



Metrics



Traces



Logs

Repeadible Builds

or “what is the problem with maven/pip/go mod”

Build Systems are programming language-centric:

- How to compile to get an executable?
- How to handle dependencies?

However:

- Common knowledge about the system is needed
- Each System behaves differently



Repeatable Builds with the help of an Independent Build System

Builds all systems the same way:

- Clear toolset leveraging from native build systems
- Easy integration into CI/CD-systems
- Common knowledge about NFAs of build systems:
 - Caching
 - Syntax
 - Workflow
 - ...

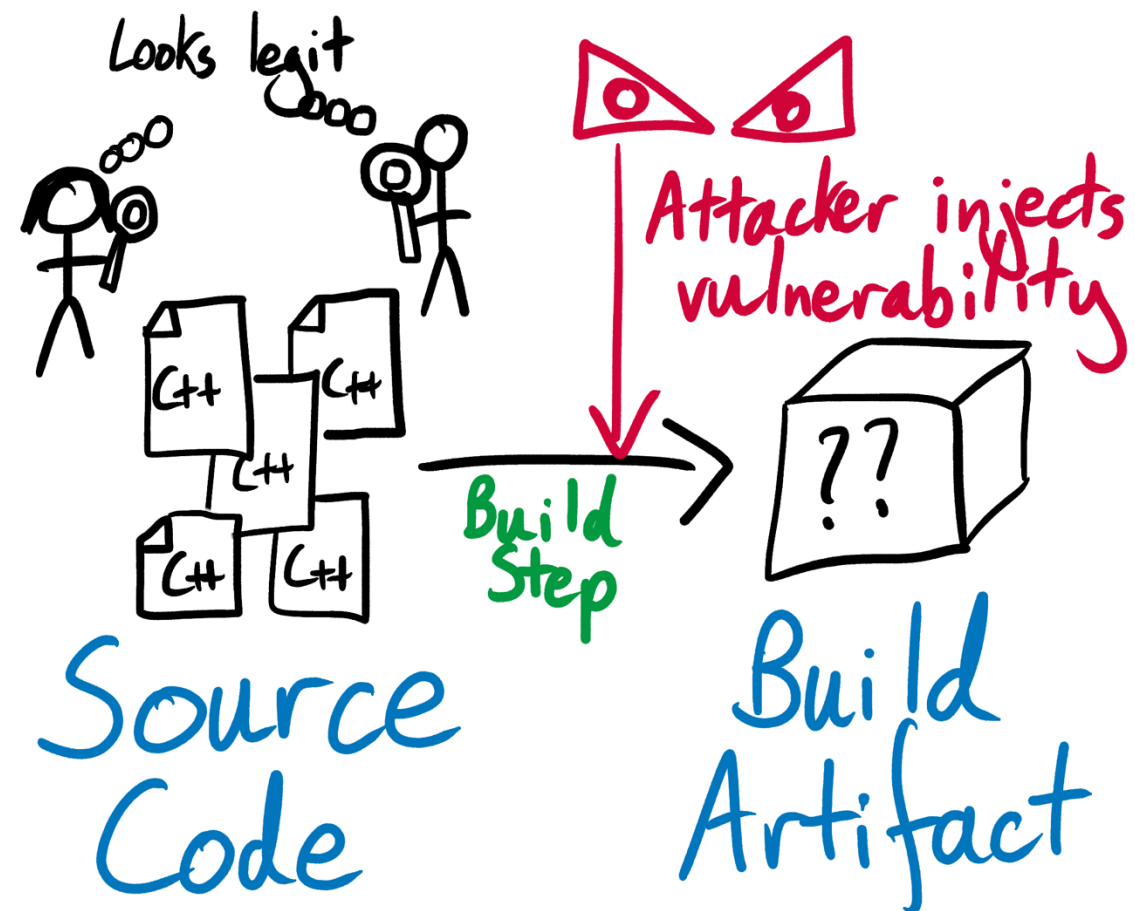
DEMO



Reproducible Builds

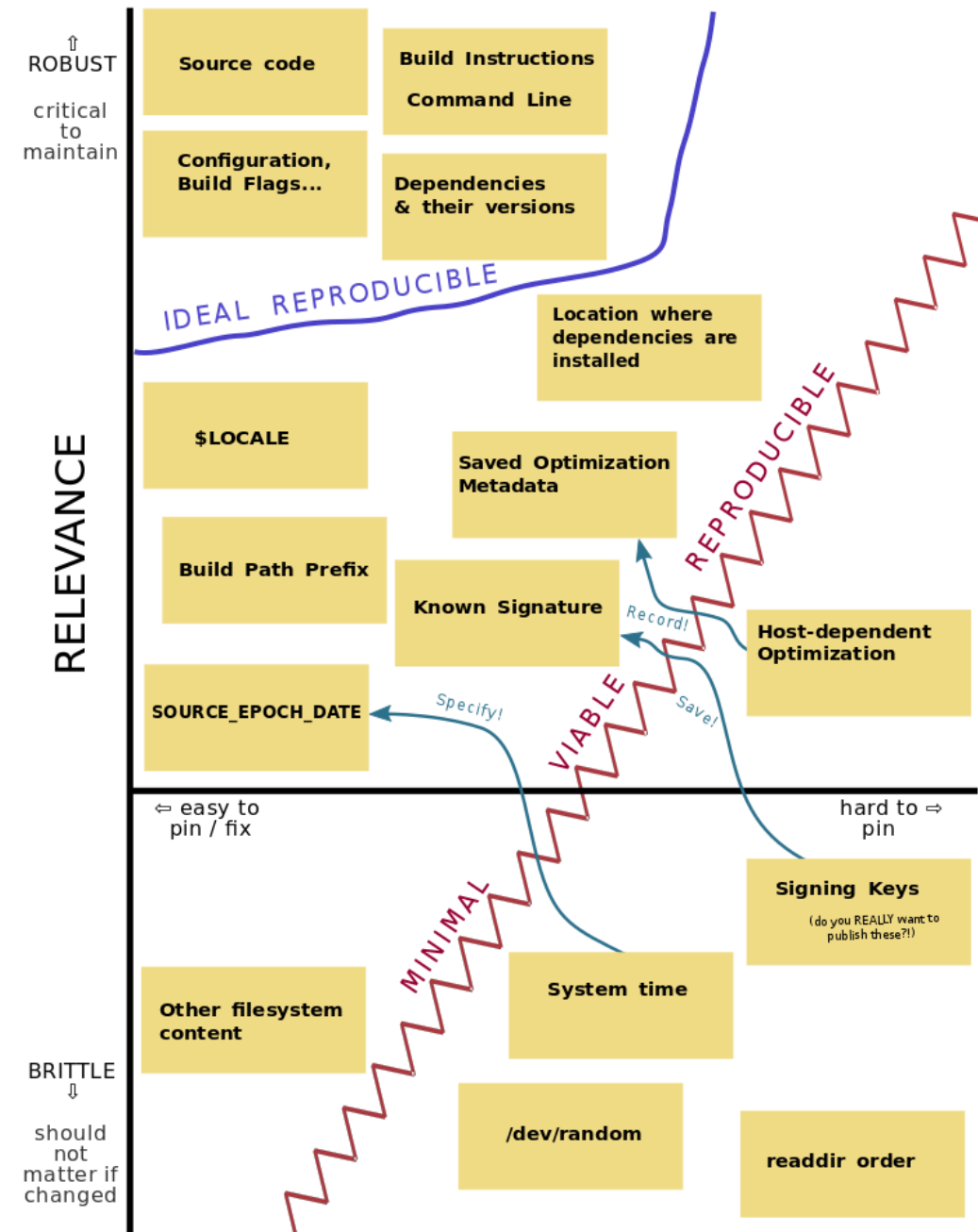
<https://reproducible-builds.org/>

“The build system should be able to take the build inputs (source code, assets, and so on) and produce repeatable artifacts. The code built from the same inputs last week should produce the same output this week”
– Google SRE Workbook

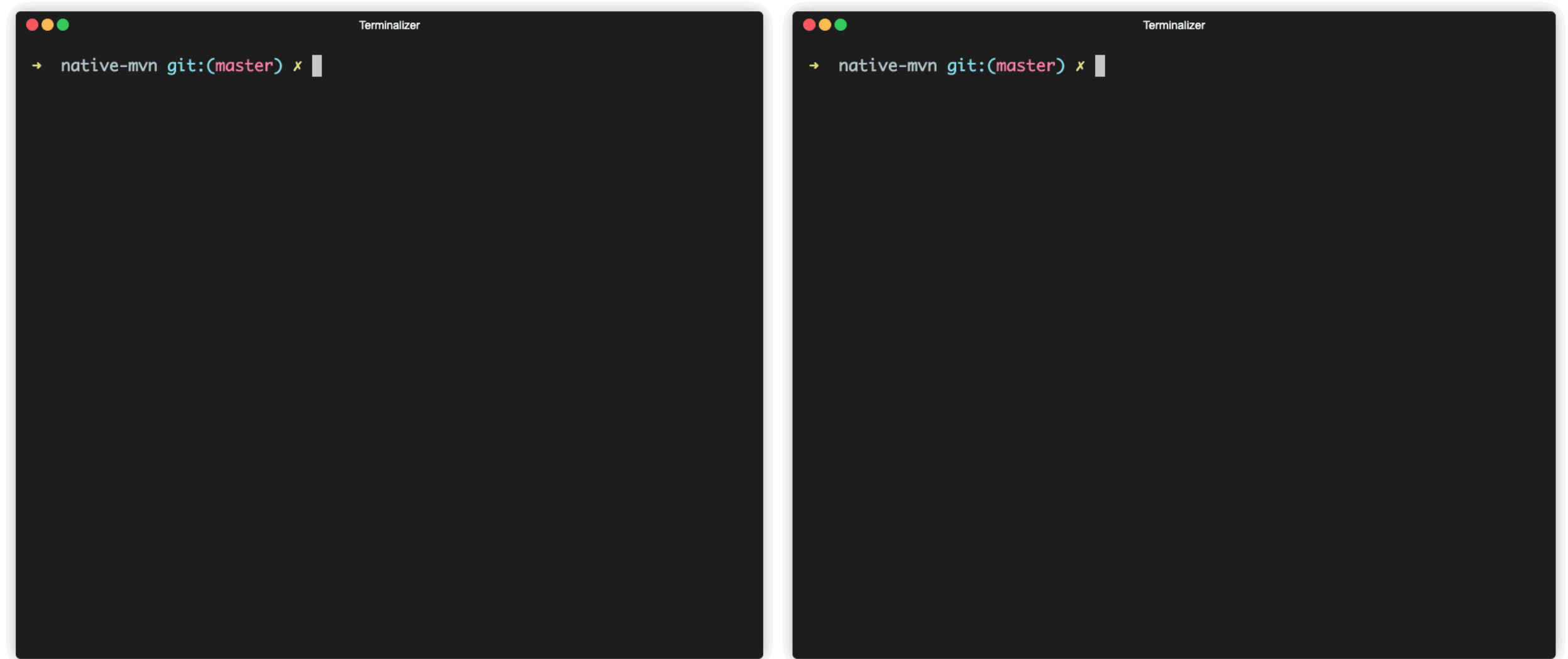


Reproducible Builds hard to achieve

Source + Build environment inputs



Example: Maven and Reproducible Builds

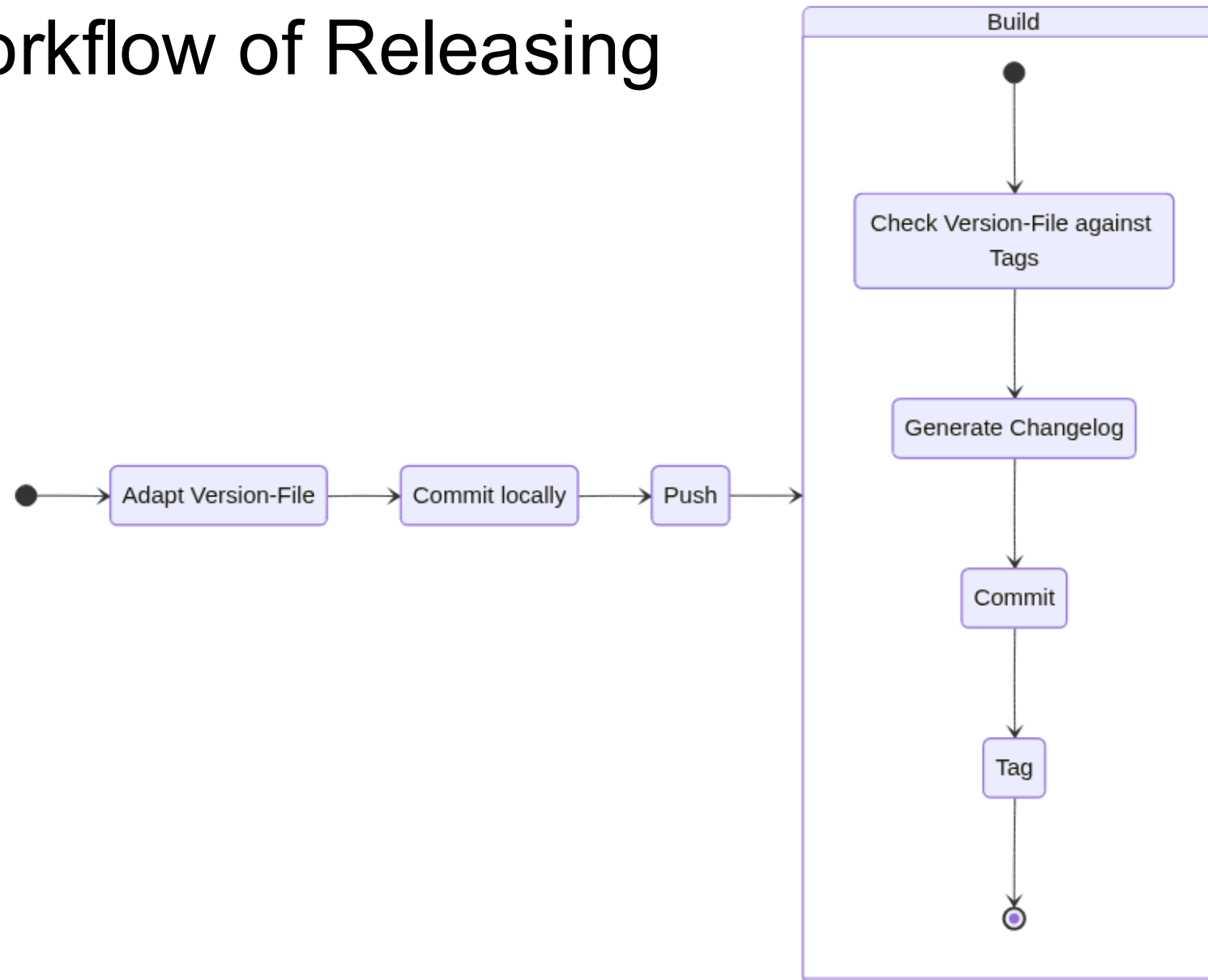


Try to get as close as possible

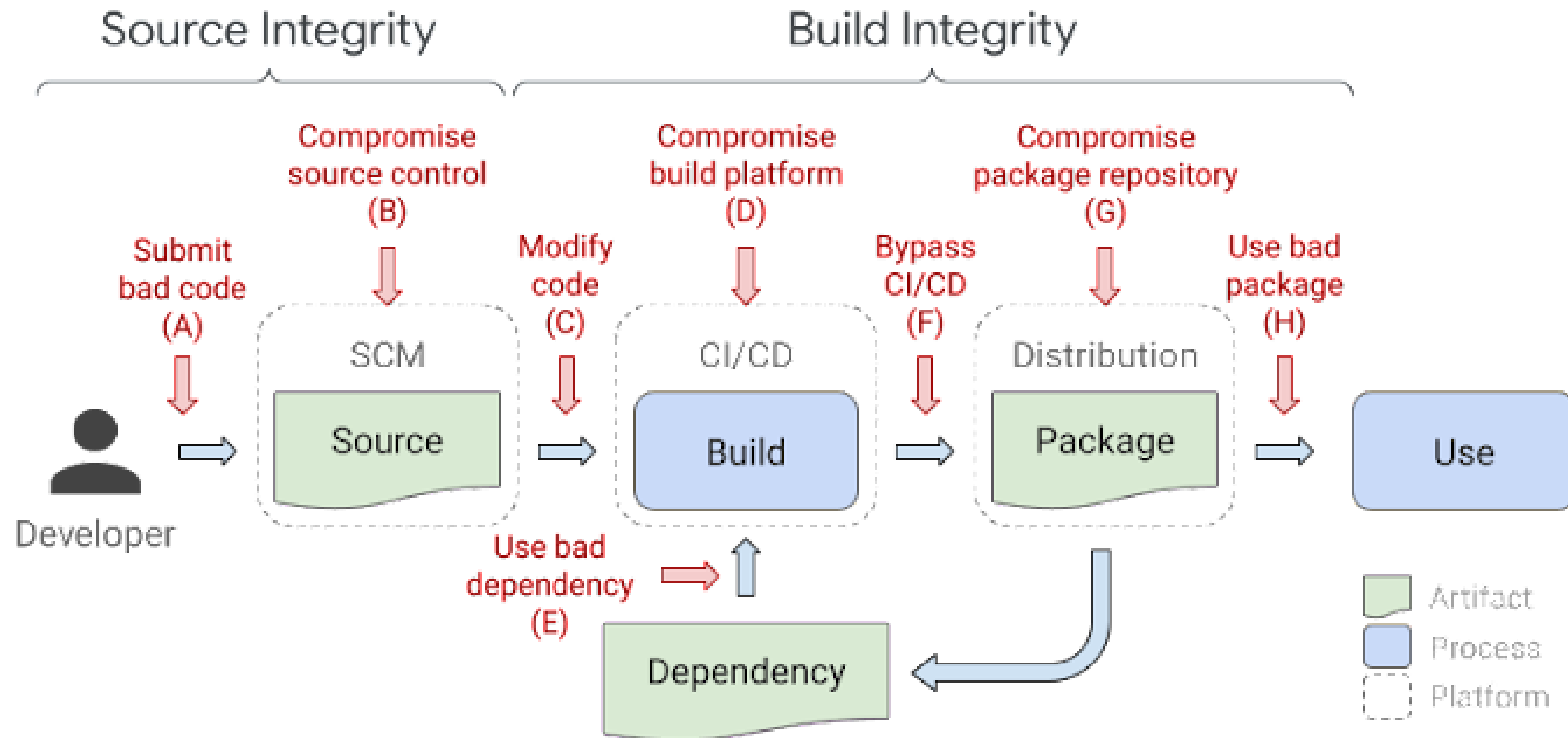
- | | | |
|----------------|--|-----------------------|
| • Sourcecode |  | • Git Tag, Commit |
| • pom.xml |  | • (Released GAV) |
| • Build System |  | • (Build Number) |
| • Docker Image |  | • Tagged
Imagename |

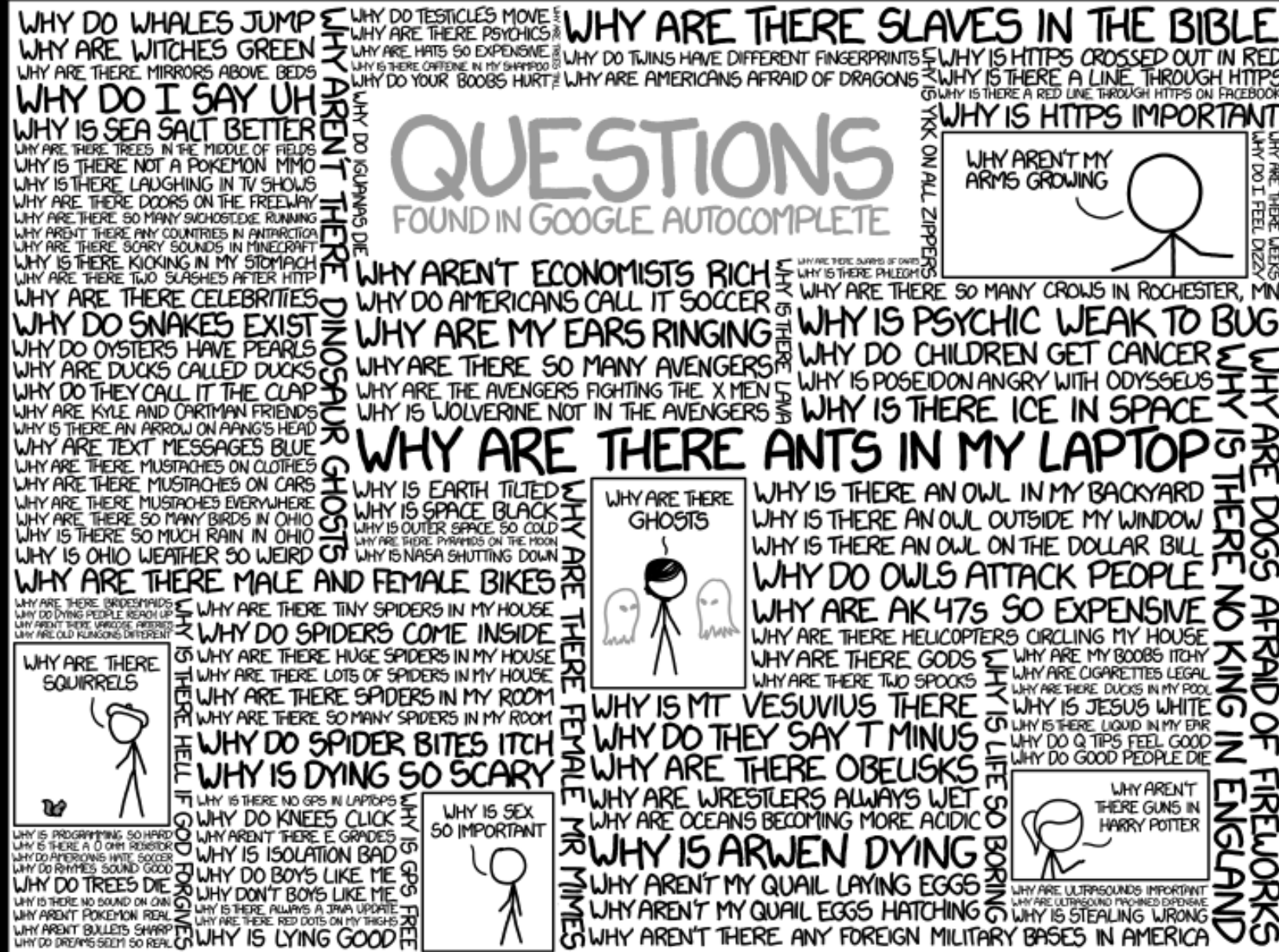
Reference only where applicable,
but always build everything with the same identifiers

Example Workflow of Releasing



Outlook for next session, SLSA





Recap Questions

1. Why are git tags important for releasing?
2. How should git tags and docker tags be mapped to each other and why?
3. What are reproducible builds?
4. Why are reproducible builds hard to achieve?
5. Why do you need Dagger / Earthly?